Relational Database Design for a Movie Management System

Soobin Jang, Jimin Lee, Yoonjoo Lee

Table of contents

01 Introduction

02 Data processing

03 Discussion

Relational Storage
ACID Compliance

04 SQL Proficiency

05 Visualization by DB

06 Conclusion

01

Introduction

Background



<Importance of Movie Data>

Vast movie data requires effective management and analysis.

<Challenges>

Unstructured data causes redundancy, integrity loss, and inefficiency.

The Purpose of Our Design



Efficient Data Management

Design a relational database to systematically store and manage movie data, ensuring data integrity and eliminating redundancy.



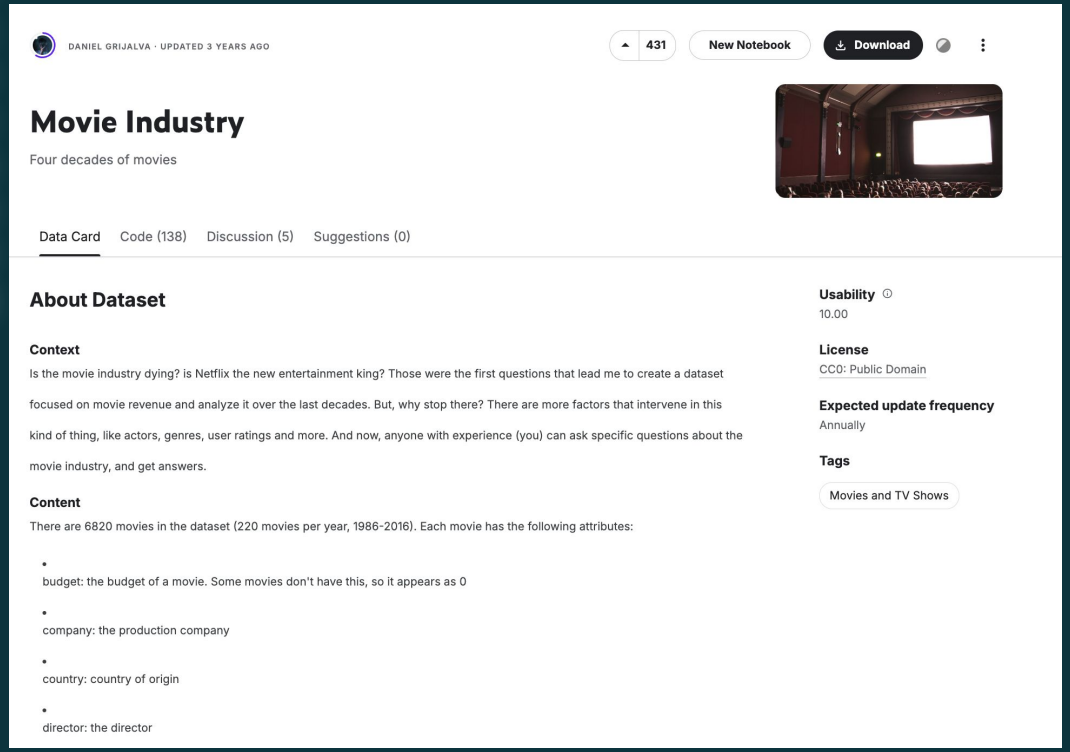
Data Analysis & Utilization

Enable effective querying and analysis of movie data using advanced SQL features, providing valuable insights for the movie industry.

02

Data Processing

Where Dataset Comes From?



DANIEL GRIJALVA · UPDATED 3 YEARS AGO

431 | New Notebook | Download

Movie Industry

Four decades of movies

Data Card | Code (138) | Discussion (5) | Suggestions (0)

About Dataset

Context

Is the movie industry dying? is Netflix the new entertainment king? Those were the first questions that lead me to create a dataset focused on movie revenue and analyze it over the last decades. But, why stop there? There are more factors that intervene in this kind of thing, like actors, genres, user ratings and more. And now, anyone with experience (you) can ask specific questions about the movie industry, and get answers.

Content

There are 6820 movies in the dataset (220 movies per year, 1986-2016). Each movie has the following attributes:

- budget: the budget of a movie. Some movies don't have this, so it appears as 0
- company: the production company
- country: country of origin
- director: the director

Usability
10.00

License
CC0: Public Domain

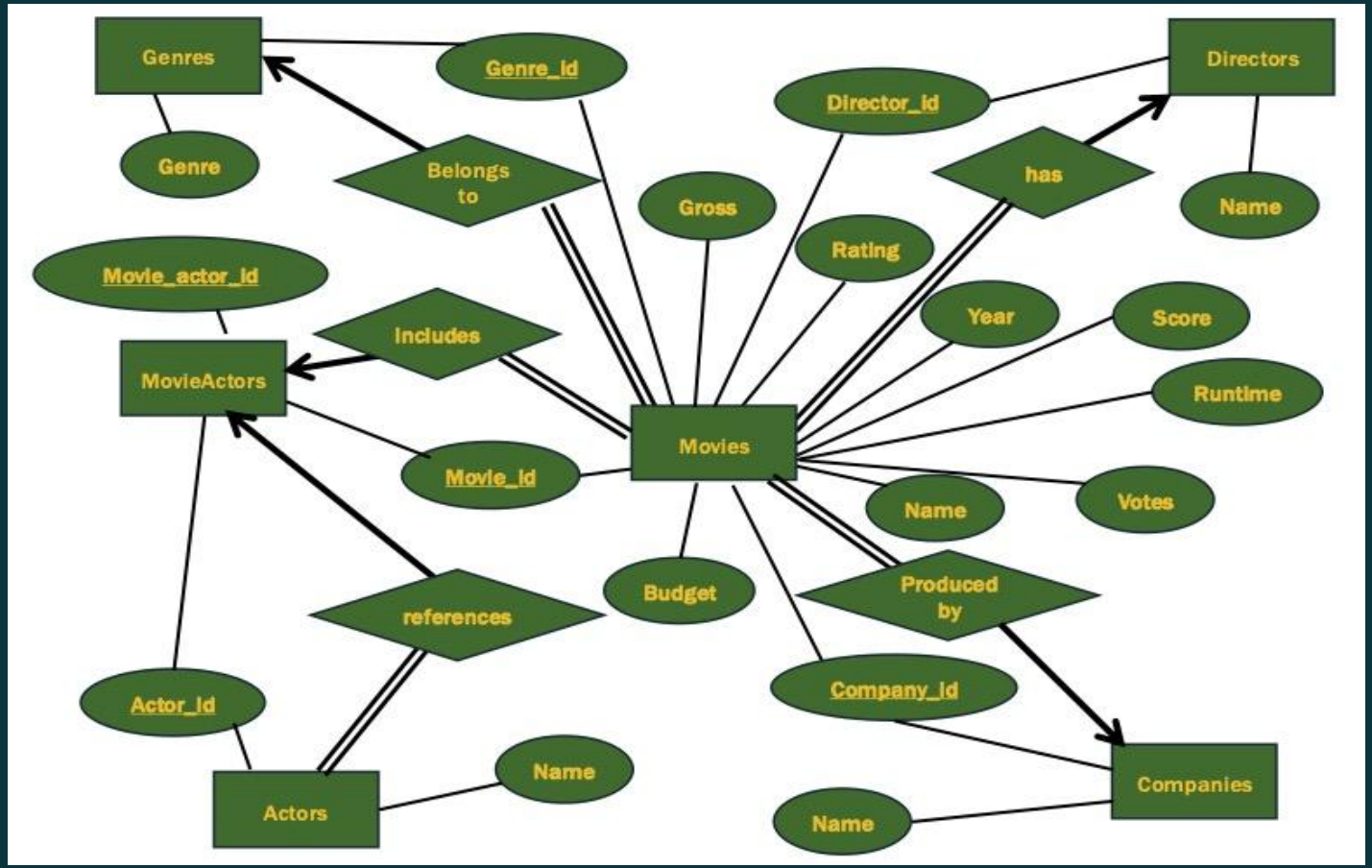
Expected update frequency
Annually

Tags
Movies and TV Shows

- Name of Movie
- Rating
- Genre
- Release Year
- Release date,
- MDb user vote
- Director
- Writer
- Actor

Link : https://www.kaggle.com/datasets/danielgrijalvas/movies

ER Diagram


Genres
Genre_Id
Directors
Director_id
Genre
Belongs to
has
Name
Gross
Rating
Movie_actor_id
Includes
Year
Score
MovieActors
Runtime
Movies
Movie_Id
Votes
Name
Budget
Produced by
references
Actor_Id
Company_Id
Name
Companies
Actors
Name

Normalization

6 tables

- Actors
- Companies
- Directors
- Genres
- Movie_Actors
- Movies



03

Discussion

Relational Storage

Relational storage organizes data into tables and defines relationships using keys (Primary/Foreign Keys)

Normalization: Reduces redundancy, ensures data is logically grouped.

Relationships: Table like Movies, Directors, and Genres are connected via Foreign Keys for consistency.

Execute SQL

```
CREATE TABLE Directors (
    Director_ID INTEGER PRIMARY KEY,
    Name TEXT NOT NULL);

CREATE TABLE Movies (
    Movie_ID INTEGER PRIMARY KEY,
    Name TEXT NOT NULL,
    Year INTEGER,
    Director_ID INTEGER,
    FOREIGN KEY (Director_ID) REFERENCES
Directors(Director_ID));
```

ACID Compliance

Ensures reliable and consistent transactions in databases.

Atomicity: All operations succeed or fail together.

Consistency: Database rules are always met.

Isolation: Transactions do not interfere with each other.

Durability: Committed changes are permanently saved.

Example in Practice

```
BEGIN TRANSACTION;

INSERT INTO Directors (Name) VALUES
('Christopher Nolan');
INSERT INTO Movies (Title, Release_Year, Rating,
Director_ID)
VALUES ('Inception', 2010, 9.0, (SELECT Director_ID
FROM Directors WHERE Name = 'Christopher
Nolan'));

COMMIT;
```

04

SQL Proficiency

String Operations

Execute SQL

```
SELECT
    Name AS Movie_Title, Year, Rating,
    Gross
FROM Movies
WHERE Name LIKE '%Star%' OR
Name LIKE '%War%'
ORDER BY Gross DESC;
```

Movie_Title	Year	Rating	Gross
Star Wars: Episode VII - The Force Awakens	2015	PG-13	2069521700.0
Avengers: Infinity War	2018	PG-13	2048359754.0
Star Wars: Episode VIII - The Last Jedi	2017	PG-13	1332698830.0
Captain America: Civil War	2016	PG-13	1153337496.0
Star Wars: Episode IX - The Rise of Skywalker	2019	PG-13	1078232589.0
Rogue One: A Star Wars Story	2016	PG-13	1056057720.0
Star Wars: Episode I - The Phantom Menace	1999	PG	1027082707.0
Star Wars: Episode III - Revenge of the Sith	2005	PG-13	868390560.0
The Chronicles of Narnia: The Lion, the Witch and the ...	2005	PG	745013115.0
Star Wars: Episode II - Attack of the Clones	2002	PG	653779970.0
War of the Worlds	2005	PG-13	603873119.0
World War Z	2013	PG-13	540455876.0
Star Wars: Episode V - The Empire Strikes Back	1980	PG	538375067.0
War for the Planet of the Apes	2017	PG-13	490719763.0

Aggregate Functions

Execute SQL

```
SELECT
    Director AS Director_Name,
    COUNT(*) AS Total_Movies,
    AVG(Score) AS Avg_Score,
    SUM(Gross) AS Total_Gross
FROM Movies
GROUP BY Director
HAVING COUNT(*) > 1
ORDER BY Total_Gross DESC, Avg_Score DESC;
```

Director_Name	Total_Movies	Avg_Score	Total_Gross
Steven Spielberg	27	7.44074074074074	9629049370.0
Anthony Russo	5	7.58	6844248984.0
Peter Jackson	11	7.74545454545455	6618045788.0
James Cameron	7	7.87142857142857	6248088578.0
Michael Bay	11	6.41818181818182	6147336610.0
David Yates	7	7.27142857142857	6021591899.0
Christopher Nolan	10	8.26	5039789178.0
J.J. Abrams	6	7.3	4659375464.0
Tim Burton	19	6.98421052631579	4456348890.0
Ron Howard	23	6.7695652173913	4312205191.0
Robert Zemeckis	17	7.24117647058824	4286713824.0
Chris Columbus	15	6.54666666666667	4133016380.0

Order by & Group by

Execute SQL

```
SELECT
  d.Name AS Director_Name,
  COUNT(m.Movie_ID) AS Movie_Count,
  SUM(m.Gross) AS Total_Gross
FROM Movies m
JOIN Directors d ON m.Director_ID =
d.Director_ID
GROUP BY d.Name
ORDER BY Total_Gross DESC;
```

Director_Name	Movie_Count	Total_Gross
Steven Spielberg	27	9629049370.0
Anthony Russo	5	6844248984.0
Peter Jackson	11	6618045788.0
James Cameron	7	6248088578.0
Michael Bay	11	6147336610.0
David Yates	7	6021591899.0
Christopher Nolan	10	5039789178.0
J.J. Abrams	6	4659375464.0
Tim Burton	19	4456348890.0
Ron Howard	23	4312205191.0
Robert Zemeckis	17	4286713824.0
Chris Columbus	15	4133016380.0

JOIN

Execute SQL

```
SELECT
  d.Name AS Director_Name,
  COUNT(m.Movie_ID) AS Movie_Count,
  SUM(m.Gross) AS Total_Gross
FROM Movies m
JOIN Directors d ON m.Director_ID =
d.Director_ID
GROUP BY d.Name
ORDER BY Total_Gross DESC;
```

actor	director	total_movies
Robert Downey Jr.	Anthony Russo	2
Robert Downey Jr.	Guy Ritchie	2
Robert Downey Jr.	Jon Favreau	2
Robert Downey Jr.	Shane Black	2
Robert Downey Jr.	David Dobkin	1
Robert Downey Jr.	Dito Montiel	1
Robert Downey Jr.	Ernest Thompson	1
Robert Downey Jr.	James Toback	1
Robert Downey Jr.	Jeremiah S. Chechik	1
Robert Downey Jr.	Joss Whedon	1
Robert Downey Jr.	Keith Gordon	1
Robert Downey Jr.	Michael Hoffman	1

HAVING

Execute SQL

```
SELECT
   m.Name AS Movie_Title,
   m.Year,
   m.Gross
FROM Movies m
WHERE m.Year >= 2015
GROUP BY m.Movie_ID
HAVING SUM(m.Gross) > 100000000
ORDER BY m.Gross DESC;
```

	Movie_Title	Year	Gross
1	Avengers: Endgame	2019	2797501328.0
2	Star Wars: Episode VII - The Force Awakens	2015	2069521700.0
3	Avengers: Infinity War	2018	2048359754.0
4	The Lion King	2019	1670727580.0
5	Jurassic World	2015	1670516444.0
6	Furious 7	2015	1515341399.0
7	Frozen II	2019	1450026933.0
8	Avengers: Age of Ultron	2015	1402809540.0
9	Black Panther	2018	1347597973.0
10	Star Wars: Episode VIII - The Last Jedi	2017	1332698830.0
11	Jurassic World: Fallen Kingdom	2018	1310466296.0
12	Beauty and the Beast	2017	1264434525.0

WITH-Clause

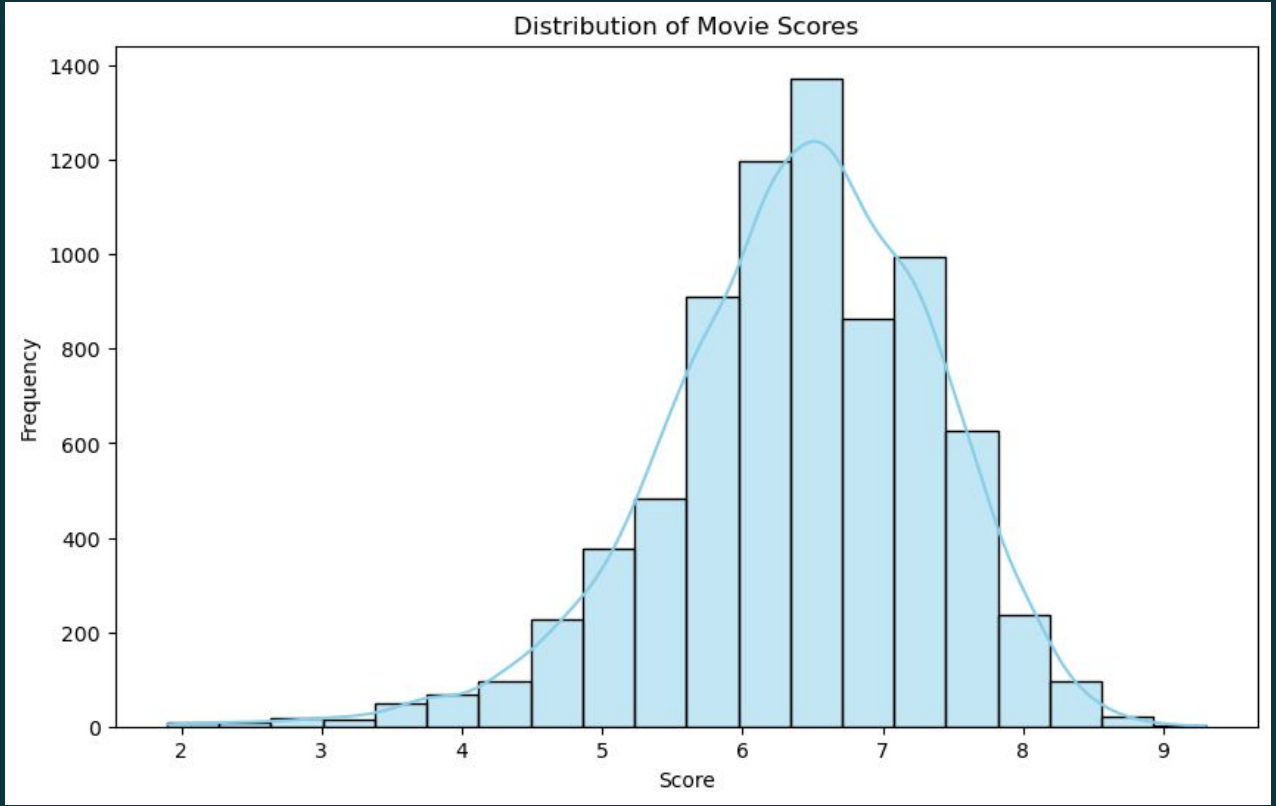
Execute SQL

```
WITH RankedMovies AS (
  SELECT
    Name, Year, Gross,
    RANK() OVER (PARTITION BY Year ORDER
BY Gross DESC) AS Rank
  FROM Movies
)
SELECT *
FROM RankedMovies
WHERE Rank <= 3
ORDER BY Year DESC, Rank ASC;
```

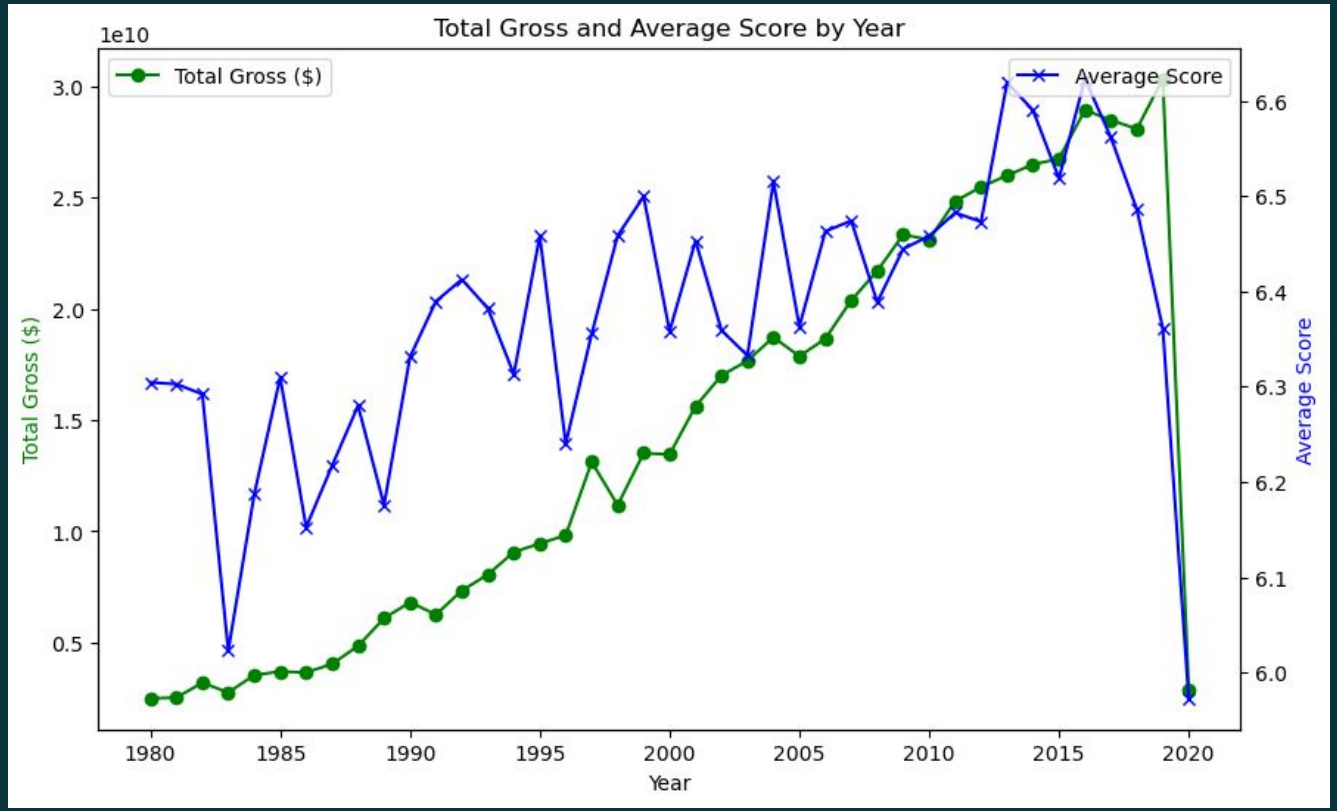
Name	Year	Gross	Rank
The Eight Hundred	2020	461421559.0	1
Bad Boys for Life	2020	426505244.0	2
Tenet	2020	363656624.0	3
Avengers: Endgame	2019	2797501328.0	1
The Lion King	2019	1670727580.0	2
Frozen II	2019	1450026933.0	3
Avengers: Infinity War	2018	2048359754.0	1
Black Panther	2018	1347597973.0	2
Jurassic World: Fallen Kingdom	2018	1310466296.0	3
Star Wars: Episode VIII - The Last Jedi	2017	1332698830.0	1
Beauty and the Beast	2017	1264434525.0	2
The Fate of the Furious	2017	1236005118.0	3

05
Visualization by DBMS

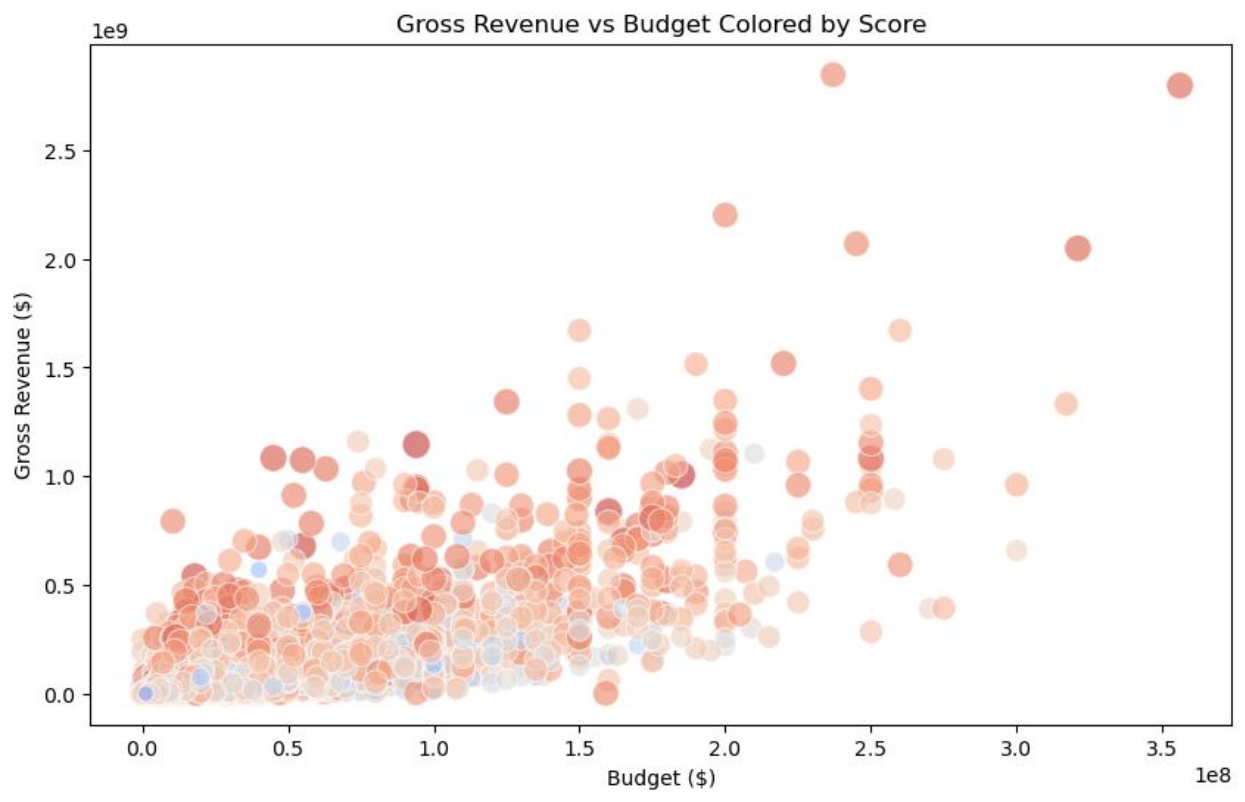
Visualization using Python



Visualization using Python



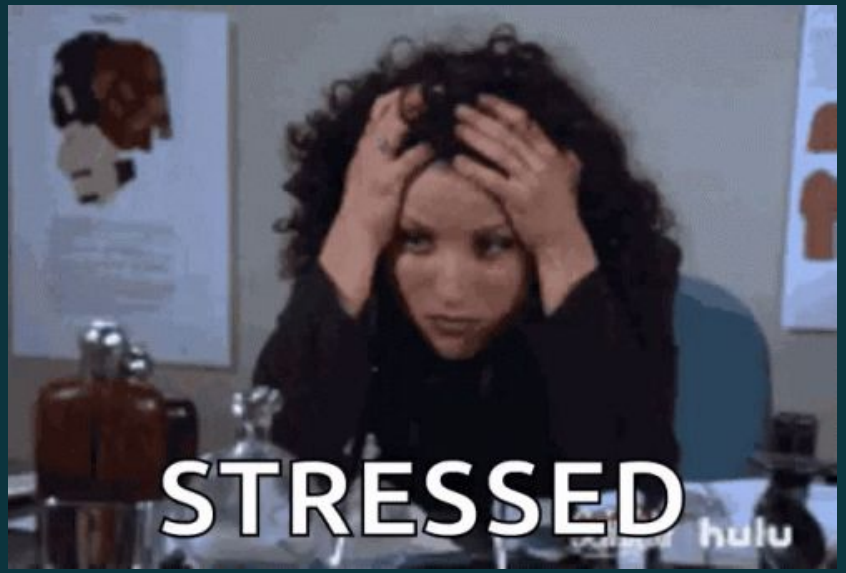
Visualization using Python



06

Conclusion

Challenge & Solution



- Challenge : missing columns in the Movies table such as Genres and Directors.

- Solution: using SQL queries to recover the Director data from the Director table and linked the Genre data through appropriate JOINS.

Conclusion



- Using SQL and Python, we analyzed and visualized movie data to extract meaningful insights.
- This project enhanced our skills in database design, querying.

Our team



Jimin Lee



Yoonjoo Lee



Soobin Jang

Thanks!

CREDITS: This presentation template was created by **Slidesgo**, and includes icons by **Flaticon**, and infographics & images by **Freepik**